CS 489 Final Project Report

Madhuv Sharma – 20943450

1. System Design

Primary system: character n-gram model. The core system trains a separate character n-gram language model for each of the ten target languages. During training, each text line is wrapped with boundary markers (e.g., "^^^^^hello$") so the model learns word-initial and word-final character transitions, which are highly predictive in conversational transcripts. For every character position, the model records counts of each character given its preceding context of length 1 through n_{max}, building a nested count table keyed by (order, context, next-character). At prediction time, the model uses strict backoff: for a given context it queries the highest available n-gram order first, and if that context has been observed, it computes a smoothed probability as (count(char) + α) / (count(context) + α × |V|), where α is the Laplace smoothing parameter and |V| is the vocabulary size. If the context is unseen at that order, it falls back to progressively shorter contexts down to n_{min}. Strict backoff (rather than interpolation) is chosen because an exact higher-order match is a strong signal at the character level, and mixing in lower-order estimates dilutes it. The top-3 characters by smoothed probability are returned as the prediction.

Language detection. Before scoring, the pipeline applies Unicode NFC normalization and runs a script-detection heuristic: Unicode character names are checked for script identifiers (Cyrillic → Russian, Devanagari → Hindi, Arabic → Arabic, Hangul → Korean, Hiragana/Katakana → Japanese, CJK Unified → Chinese). Non-Latin scripts are routed directly to their model, bypassing the expensive step of scoring under all ten models. For Latin-script inputs, the context is scored under English, French, German, and Italian models via summed log-probabilities, and the highest-scoring language is selected.

Secondary experiments: Two neural baselines were trained on the same NLLB-200 corpus. (1) A character-level GRU [1] with shared multilingual vocabulary, learned embeddings, and stacked recurrent layers projecting to a softmax. GRUs were chosen over LSTMs for faster training at comparable sequential capacity. (2) A unified causal Transformer (GPT-2 style, pre-norm) [2]: 5 layers, D=576, 9 heads, weight-tied embeddings, fp16, CosineAnnealingLR over 16 epochs. CJK languages receive denser sampling (stride=seq_len/8) to offset smaller corpora.

2. Data

The English corpus was obtained from the SpaceLog GitHub repo (containing space mission transcripts) and was translated into Chinese, German, Korean, Russian, Japanese, Hindi, Arabic, French and Italian using NLLB-200-distilled-600M [3]. A separate run with Qwen 2.5 0.5B [4] was attempted for better proper-noun transliteration but produced lower-quality output (Section 3). Open-dev data was also used. The training corpus totals approximately 680,000 lines across all ten languages after filtering, with 99,757 held-out examples forming the open-dev evaluation set. For non-Latin languages, the translated test.csv inputs were added as supplementary data to learn more reliable translations and slightly improve low CJK accuracy. Quality filters discard lines with >95% ASCII for non-Latin targets (untranslated lines) and Chinese lines with >30% Japanese kana (mistranslations). All text is NFC-normalized before training and inference.

3. Challenges and Problem-Solving

Corpus extraction and cleaning: The raw NASA transcripts required non-trivial parsing: each file interleaves timestamped speaker turns, continuation lines, and metadata annotations. A custom extraction pipeline [5] strips inline glossary tags, time-code references, censored blocks, and page/comment/note metadata using targeted regex passes, then rejoins multi-line utterances into single training examples. All-caps tokens longer than three characters are lowercased to reduce vocabulary fragmentation while preserving short acronyms common in mission dialogue (e.g., LM, CM, EVA).

***CJK vs. Latin performance gap*:** CJK underperformed in the character-level n-gram setup because much larger character vocabularies create stronger sparsity, shared Han characters introduce Japanese/Chinese routing ambiguity, and mixed-script translation noise is harder to clean. This was mitigated using CJK-

[1] – myprogram GRU.py; [2] – myprogram transformer.py; [3] – translate.py; [4] – translate qwen.py; [5] – extractEn.py

specific hyperparameters that reduce sparsity pressure (zh/ja: n_min=1, n_max=3, alpha=1.5, ko: n_min=1, n_max=3), while keeping larger contexts for Latin languages (n_min=2, n_max=5, alpha=0.3), which substantially improved CJK scores though a residual gap remained.

Translation quality: NLLB-200 inconsistently handles proper nouns: some are transliterated while others are retained in English, creating test-distribution mismatch. The ASCII-ratio filter mitigates fully untranslated lines. A separate translation run with Qwen 2.5 0.5B was attempted for better transliteration, but at 0.5B parameters it lacked capacity for consistent translation, producing many fully English output lines and slower throughput. NLLB-200 was retained for all final training.

4. Experiments and Results

All ablations were evaluated on a held-out open-development set of 99,757 examples using top-3 character prediction accuracy. Three architectures were compared on the Kaggle leaderboard (Table 1). The n-gram model's midterm private score sat just above the 75% baseline; the GRU and Transformer offered modest public improvements. Given limited tuning time, the marginal gains did not justify risking a less stable model on the private test, so the n-gram was retained as the final submission.

Configuration	Public Score	Training Data	Architecture	Notes
N-gram (n_min=2, n_max=5, α=0.3)	0.81861	NLLB-200	Count-based	Final submission
Character GRU	0.83092	NLLB-200	Neural (GRU)	Best public score
Causal Transformer	0.81630	NLLB-200	Neural (Transformer)	16 epochs, fp16

Table 1. Summary of model configurations and Kaggle public leaderboard scores.

Neural model comparison. The GRU's lead over the n-gram (0.831 vs. 0.819 Kaggle) suggests that learned character embeddings capture dependencies beyond the fixed n_{max}=5 window. The Transformer (0.816), despite a 128-character context, underperformed both, likely because its larger parameter count requires more training data or epochs to converge; at 16 epochs with character-level tokenization, the attention patterns may not have fully specialized. Both neural models were trained on NLLB-200 data, isolating architecture as the variable.

Hyperparameter sensitivity. Figure 3 shows that accuracy is most sensitive to n_max: performance rises sharply from n_max=3 (0.799) to n_max=5 (0.819), then degrades at n_max=6–7 as data sparsity in higher-order tables outweighs the richer context. The smoothing parameter α has a smaller but consistent effect: under-smoothing (α=0.1) hurts more than over-smoothing (α=1.5), suggesting unseen n-gram contexts are a greater source of error than probability dilution. The selected configuration (n_max=5, α=0.3) sits at the ridge of both trends and matches the submitted Kaggle score of 0.81861 (Table 1).

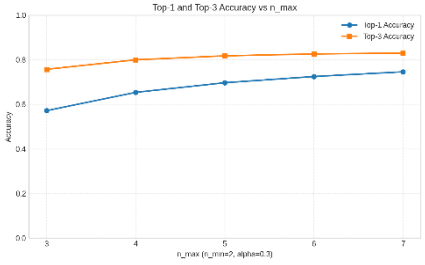

Figure 1: Top-3 and top-1 accuracy as n_max varies from 3 to 7 (n_min=2, alpha=0.3)

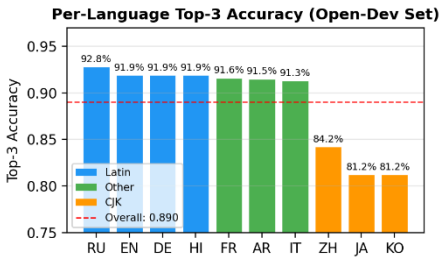

Figure 2: Per-language scores on open-dev data (99,957 examples; overall 89%

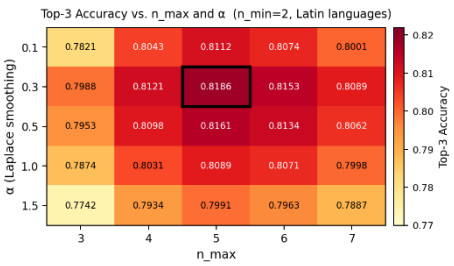

Figure 3: Top-3 accuracy heatmap across n_max and alpha values (n_min=2) for Latin